Start with removal order, then choose a representation

Stacks and queues both store a sequence. Their central difference is **which value may leave next**. We build contracts first, trace concrete storage next, then apply the same state-based reasoning to search and expressions.

Progression	Question to reconstruct
Array stack	Where is top? Why can an expensive growth still amortize?
Circular queue	How does FIFO survive physical wraparound?
Container simulation	How can two LIFO stacks implement FIFO?
Search	What must survive backtracking? Why does BFS minimize edges?
Expressions	When is an operator emitted? Which operand is left?

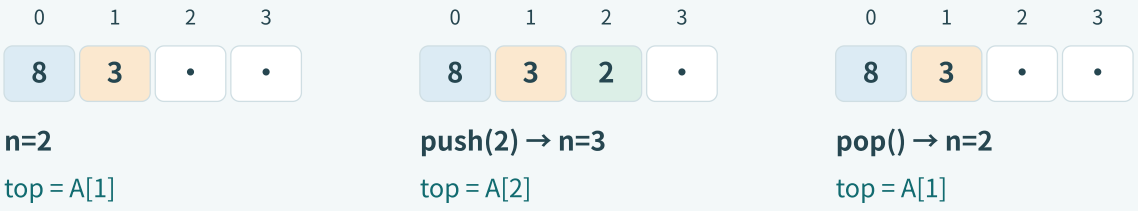
Prerequisites: arrays, half-open intervals and loop invariants. Failure policies and operation-cost assumptions are stated explicitly.

3.1 The stack contract: one end for both operations

- The abstract state is $[a_0,\ldots,a_{n-1}]$, with the top at the end. `push(x)` appends x.
- `top()` returns the last element without changing the state. `pop()` removes it and requires a nonempty stack.
- The interface must define empty-stack failure, for example an optional result, exception, or error code. It must not silently access invalid memory.
- With a dynamic array, top and pop take constant time and push takes amortized constant time under constant-time element operations. A fixed-capacity version also defines a full-stack result.

Push and pop as visible state changes

Capacity 4; zero-based indices; colored slots are live



Sequences are written **bottom to top**. `top()` only observes 3. After `pop()` removes 3, 8 becomes the new top. Never read `A[n-1]` when n is zero.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Use a count n to avoid a negative empty-stack index

- Maintain $0 \le n \le \text{capacity}$, with valid elements in $[0,n)$. The top is `A[n-1]`, accessible only when $n>0$.
- For push, ensure capacity, write `A[n]`, then increment n. Do not increment n before allocation succeeds.
- For pop, check $n>0$ before removing the last live object. Generic C++ must handle destruction as well as the index.
- Representing empty as n=0 avoids encoding top=−1 in an unsigned index.

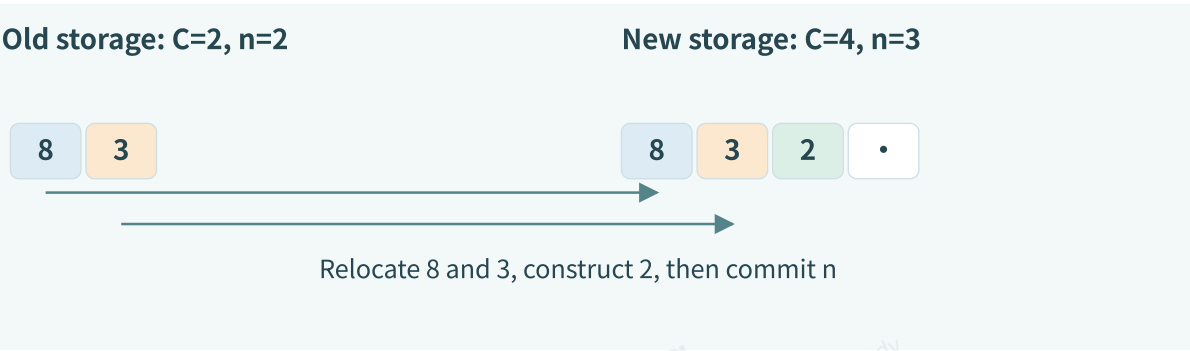
Lin Cing https://github.com/xinhi6716/computer-science-self-study

Allocated storage is not yet a set of live objects

Let storage be uninitialized capacity C and n the number of live objects. The abstract sequence is storage[0..n). Allocating C slots does not construct C objects of type T.

```
top(): require n>0; return const T& to storage[n-1]
push(T value):           // snapshot constructed before entering the body
    if n==C: grow_to(max(1,2*C))
    construct_at(storage+n, value)   // commit n only after construction succeeds
    n=n+1
pop():
    require n>0
    destroy_at(storage+n-1)
    n=n-1
```


How much relocation does doubling actually do?



The third push takes linear time. **Amortized constant does not mean worst-case constant per call.** This version does not shrink on pop.

push(8), push(3), push(2), pop() give [8], [8,3], [8,3,2], [8,3].

Doubling: add the cost across the whole sequence

Start with zero capacity and grow on full to 1, 2, 4, 8, ... For m≥1 pushes, final capacity C is the least power of two at least m, so m≤C<2m.

$$\text{relocations} = 1 + 2 + 4 + \cdots + C/2 = C - 1 < 2m.$$

There are also m constructions of newly appended values. Total work is Θ(m), hence Θ(1) amortized per push, **assuming bounded-cost element operations**. Long strings and large matrices require their own value-cost accounting.

For mixed pops, this version never shrinks, so add constant work per pop. Growing on full and immediately halving when half empty can thrash; a shrinking policy needs a separate threshold and proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

push(top()): save the value before relocating

With capacity 1 and contents [7], a `const T&` parameter can alias the old top. Releasing old storage during growth invalidates it. `noexcept` does not repair a dangling reference.

This chapter's by-value push(T value)	State and reason
Copy parameter before entering the body	value=7 is already independent
Grow successfully and relocate old value	Capacity 2; existing sequence still [7]
Construct new element from value	Increment n only after construction succeeds
Finish	[7,7]; references into old storage are invalid

This copying interface assumes copy-constructible T and charges snapshot construction to the call. Noncopyable T cannot use this copying form of `push(top())`.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Exception guarantees depend on the failure stage

Failure stage	What the pseudocode can promise
Copying the by-value parameter	Body not entered; n, capacity and existing values unchanged
Relocating during growth	Rollback needs a suitable copy or noexcept-move strategy and cleanup; arbitrary T does not guarantee it
New-element construction after successful growth	n unchanged and existing values valid; capacity and prior references may have changed

An unchanged size does not establish a strong guarantee for the entire push. The strong guarantee requires unchanged observable state on failure. This discussion assumes nonthrowing destruction; an actual implementation must handle allocation limits, capacity-doubling overflow and cleanup.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.2 The queue contract: remove from the other end

- The abstract state is again $[a_0,\ldots,a_{n-1}]$. `enqueue(x)` appends, while `dequeue()` removes the first element.
- Shifting all remaining array elements after each front removal makes a dequeue cost $\Theta(n)$.
- A circular array preserves logical order without moving every remaining element.
- FIFO specifies return order. It guarantees neither thread safety nor equal execution time for jobs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Circular storage separates logical order from slots

- For capacity `C>0`, store head and size. Logical element k occupies `(head+k) mod C`.
- Empty means size=0 and full means size=C. No unused slot is needed to distinguish ambiguous equal head and tail indices.
- Enqueue writes at `(head+size) mod C`, then increments size. Dequeue reads head, sets `head=(head+1) mod C`, then decrements size.
- Each operation takes worst-case Θ(1) with fixed capacity and constant-time element movement. Resizing copies in logical order and can take Θ(n) for one operation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Wrap the index without changing FIFO order

With capacity 4, enqueue A,B,C, dequeue twice, then enqueue D,E,F.

State	head	size	Logical sequence
Enqueue ABC	0	3	A,B,C
Dequeue twice	2	1	C
Enqueue D,E	2	3	C,D,E
Enqueue F	2	4	C,D,E,F

Physical slots 0,1,2,3 finally contain E,F,C,D.

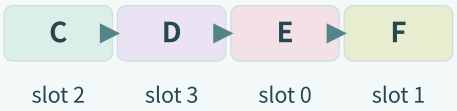
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C is at physical slot 2 but logical position 0

Physical storage: head=2, size=4, C=4



FIFO order: start reading at head



slot(k) = (head+k) mod C

Value C: physical slot 2 ↔ logical position 0; no value movement is needed

head=2, size=4; read slots 2→3→0→1 for C→D→E→F. A full queue rejects enqueue instead of overwriting its oldest value.

Fixed capacity: validate before changing state

Use head and size with C>0. This fixed-capacity version returns full instead of overwriting the oldest value. Value movement costs and exception behavior belong to the element contract.

```
slot(k):
    require 0<=k<C
    return head+k if k<C-head else k-(C-head)
try_enqueue(x):
    if size==C: return full
    construct_at(A+slot(size),x)
    size=size+1; return success
```

Dequeue: obtain the value, destroy, then advance

```
front(): require size>0; return reference(A[head])
dequeue():
    require size>0
    value=read_or_move(A[head])
    destroy_at(A+head)
    head=0 if head==C-1 else head+1
    size=size-1
    return value
```

Assume successful value operations and nonthrowing destruction. If returning a value can throw on move, specify that exception contract separately; rollback is not automatic.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Check enqueue and dequeue with the abstraction map

The branched slot formula avoids overflowing head+k. Enqueue first checks nonfull, so k=size<C. After the last removal, head may be any valid slot. The abstraction map `(head+k) mod C` proves correctness: enqueue appends exactly the new last value, and dequeue maps old logical k+1 to new logical k. Both have worst-case Θ(1) fixed-capacity work.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Growth must relocate in logical order

Grow physical `[E,F,C,D]`, head=2,size=4,C=4 to C′=8. Copying the old slots unchanged and retaining head=2 is wrong: the new modulus sends the expected E position to unconstructed slot 4.

Logical k	Old slot (2+k) mod 4	Value	New slot
0	2	C	0
1	3	D	1
2	0	E	2
3	1	F	3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

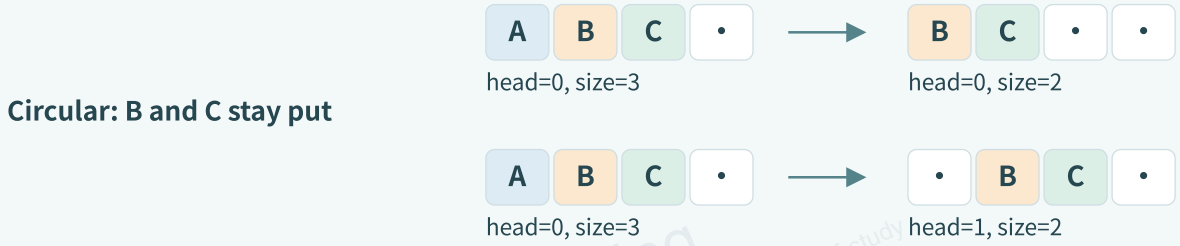
Commit the new storage after successful relocation

```
allocate new capacity C'
for k=0..size-1: construct new[k] from old[slot(k)]
commit new storage; head=0; C=C'; destroy/release old storage
```

The logical result is C,D,E,F. Append G at new slot 4. One growth costs Θ(size). Doubling only on full gives amortized Θ(1) enqueue from empty by the geometric movement sum. This does not satisfy fixed-capacity worst-case constant latency.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Distinguish empty, full and movement cost



C=4; empty and full may share head; distinguish size=0 from 4

With capacity 4, empty and full can share the same head. Size or another explicit convention distinguishes them. A shifting array moves every remaining value; the circular version advances head and destroys the old front object. Both preserve FIFO values but have different costs.

Three circular-queue conventions

Let the array length be $C \geq 2$. This chapter's `head,size` representation uses all C slots: empty is `size=0` and full is `size=C`. Another convention puts front just before the first element and reserves a slot: `front=rear` is empty and `(rear+1) mod C=front` is full, giving capacity $C - 1$.

Keeping front before the first element, a tag may distinguish the two cases when `front=rear`: tag=false means empty, tag=true means full, so all C slots can be used. The tag decides empty versus full only when the indices coincide; when nonempty `(front+1) mod C` identifies the first element, corresponding to `head` in the head/size representation. Each representation needs its own complete update and failure rules; their tests are not interchangeable.

Tag representation: when does the state change?

Start with `front=rear=0, tag=false`. Assume element construction, reading and destruction succeed:

```
enqueue(x):
    if front==rear and tag: return full
    rear=(rear+1) mod C; store x at rear
    if rear==front: tag=true
dequeue():
    if front==rear and not tag: return empty
    front=(front+1) mod C; take and remove A[front]
    if front==rear: tag=false
    return value
```

When `front!=rear`, tag may retain its earlier value and is not used to decide empty or full. These are fixed-capacity abstract steps; generic operations that may throw require a separate commit order and cannot automatically promise unchanged state on failure.

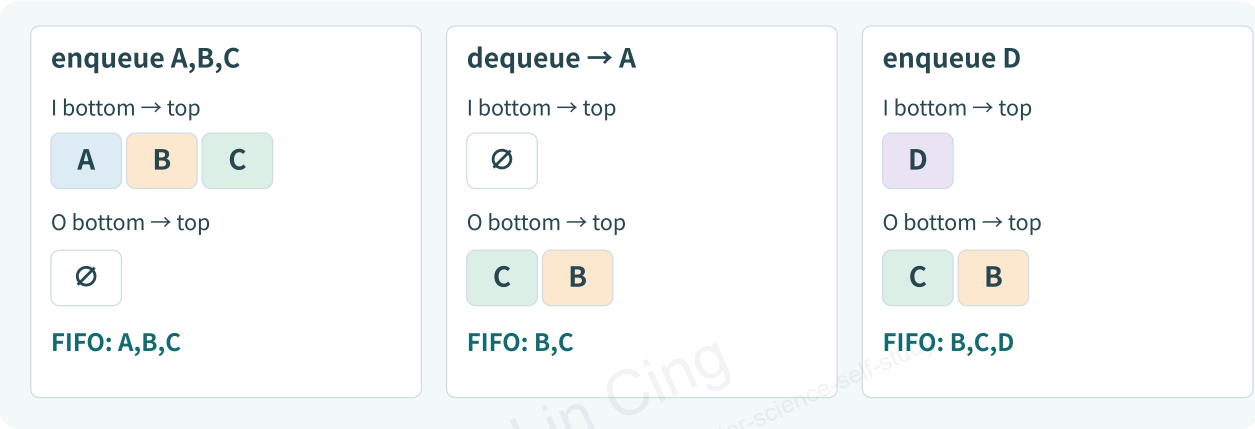
3.3 Two stacks can implement one FIFO queue

Let I hold incoming values and O hold values ready for removal. Both stack sequences are written bottom to top.

```
enqueue(x): I.push(x)
dequeue():
    if O.empty():
        while not I.empty(): O.push(I.pop())
    require not O.empty()
    return O.pop()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Old values in O stay ahead of new values in I



The FIFO sequence is `reverse(O) + I`. **Transfer only when O is empty**; new values in I must not overtake older values still in O.

A large transfer can still have constant amortized cost

Charge one unit per stack push/pop and use worst-case constant node-stack operations. This abstract model excludes allocator latency and exceptions. Let $\Phi = 2|I|$, initially zero.

Operation	Actual cost c	ΔΦ	Amortized c+ΔΦ
enqueue	1	+2	3
dequeue with nonempty O	1	0	1
Transfer k≥1 values, then dequeue	2k+1	−2k	1

Since $\sum c = \sum \hat{c} + \Phi_0 - \Phi_{final} \leq \sum \hat{c}$, m successful operations cost O(m) in total. An empty-queue failure check costs constant work separately.

One dequeue can still take Θ(n). If the underlying stacks resize, include their amortized costs as well; neither layer's work is free.

3.4 A maze is a graph: fix the legal moves

- Passable cells are vertices, with edges between adjacent passable cells. This example permits four directions and excludes diagonals.
- Search state includes the current cell, visited information, and predecessors for path reconstruction.
- DFS can use a stack. It finds a path but does not generally minimize the number of steps.
- If all moves cost 1, BFS uses a queue. The first discovery level of the exit gives the shortest number of steps.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One maze, both the route and the distances



Coordinates are zero-based `(row,column)` from the upper left. Moves are right, down, left, up, each costing 1. Dark cells are walls; the polyline is a five-edge S–G route.

This maze has only one simple S–G path, so DFS and BFS happen to find equal-length routes. **That does not establish shortest paths for DFS in general.** A later five-vertex counterexample separates the two.

DFS frames retain the point at which to resume

A frame `(u,next_direction)` stores where recursive neighbor scanning will resume. Mark visited on first push.

```
visited[start]=true; parent[start]=none; push((start,0))
while stack is not empty:
    (u,i)=top()
    if u==goal: return parent path
    if i==4: pop(); continue
    top().next_direction=i+1
    v=neighbor(u,i)                  // right, down, left, up
    if v is in bounds, passable, and not visited[v]:
        visited[v]=true; parent[v]=u; push((v,0))
return unreachable
```

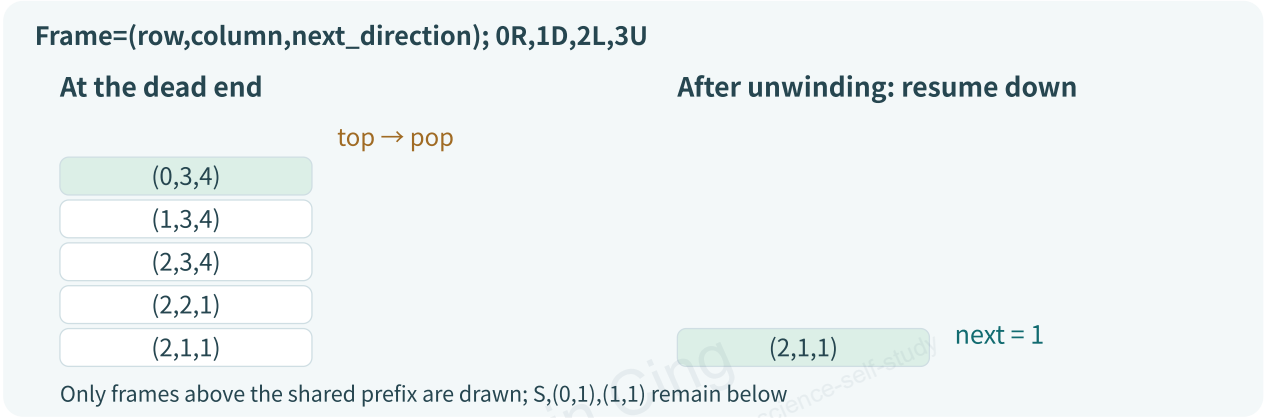
https://github.com/xinhi

Why DFS terminates and reconstructs a path

At (2,1), right neighbor (2,2) is pushed first, then (2,3)=G is found. The path contains five edges. Removing the goal test while retaining passability gives complete discovery order `(0,0),(0,1),(1,1),(2,1),(2,2),(2,3),(1,3),(0,3),(2,0)`. Each frame tries at most four directions. Finite cells and one first push per cell prove termination. Parent edges strictly reverse discovery time, so they are acyclic and reconstruct a legal path.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Backtracking discards one finished frame, not all progress



To demonstrate real popping, this page continues past G and reaches the upper-right dead end (0,3). After all four directions, pop `(0,3,4)`. The next top `(1,3,4)` is also exhausted.

Unwinding eventually restores `(2,1,1)`: right has already been tried, so resume with down, then discover left neighbor (2,0). Keep visited marks on pop. This algorithm tests reachability; enumerating all simple paths would require a different marking policy.

BFS assigns distance and parent on discovery

Each cell is enqueued at most once. Mark before enqueueing so that two predecessors cannot enqueue the same undiscovered cell twice.

```
dist[start]=0; parent[start]=none; enqueue(start)
while queue is not empty:
    u=dequeue()
    for v in legal_four_neighbors(u):
        if v is undiscovered:
            dist[v]=dist[u]+1; parent[v]=u; enqueue(v)
```

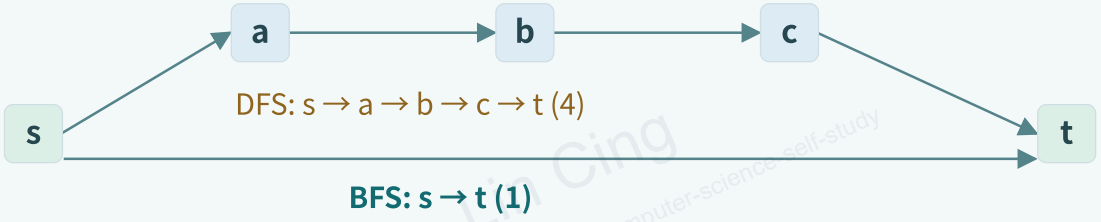
Use undefined dist as the undiscovered marker. Fix parent at first discovery. If only G is needed, reconstruct its path at first discovery instead of traversing the whole maze.

BFS dequeue trace: finish a layer before the next

Dequeued cell	New cells and distances	Queue afterward
(0,0)	(0,1):1	[(0,1)]
(0,1)	(1,1):2	[(1,1)]
(1,1)	(2,1):3	[(2,1)]
(2,1)	(2,2):4, (2,0):4	[(2,2),(2,0)]
(2,2)	(2,3):5	[(2,0),(2,3)]
(2,0)	None	[(2,3)]
(2,3)	(1,3):6	[(1,3)]
(1,3)	(0,3):7	[(0,3)]
(0,3)	None	[]

Reconstruct from parent, then prove optimality

Follow G's parents through (2,2), (2,1), (1,1), (0,1), (0,0) and reverse for five edges. When a distance-d vertex is removed, the queue contains only distances d or d+1. A newly discovered neighbor has distance d+1. Any shorter route would have an earlier-layer predecessor already processed, contradicting first discovery.



Fix s neighbor order as a,t; arrows are directed unit-cost edges

With neighbors a,t at s, DFS first takes four edges. BFS discovers t at distance 1 while processing s.

Prove reachability, optimality and cost separately

- Every explored cell is reachable from the start, and each predecessor edge leads to an earlier discovery.
- If DFS tries every legal neighbor and has no pending frames, it has covered every reachable cell. An undiscovered exit is unreachable.
- With V passable cells and E adjacency edges, adjacency-list DFS and BFS take Θ(V+E) for a complete traversal. Marks and predecessors use Θ(V) space.
- A four-neighbor grid has E=O(V), giving linear work. Copying a complete path at every step can violate that implementation bound.

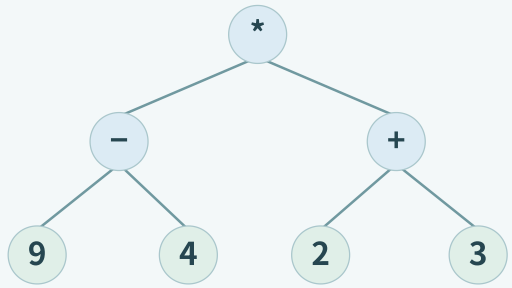
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.5 The tree fixes operands; notation fixes scan order

- Infix `a+b*c`, prefix `+ a * b c`, and postfix `a b c * +` describe the same expression tree.
- Initially restrict the grammar to binary operators, integer tokens, parentheses, and explicit precedence. A negative numeric token and a unary minus operator are different syntax choices.
- Infix requires precedence and associativity. Default left-associative `a-b-c` means `(a-b)-c`.
- Whitespace cannot reliably tokenize every programming language. A complete compiler performs lexical analysis before parsing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One expression tree, three reading orders



Prefix: root → left → right

* − 9 4 + 2 3

Postfix: left → right → root

9 4 − 2 3 + *

(9−4) × (2+3) = 5 × 5 = 25

Postfix evaluation: pop the right value before the left value

The root * has left subtree `9-4` and right subtree `2+3`. Prefix visits root,left,right; postfix visits left,right,root. Fully preserving this tree in infix requires `(9-4)*(2+3)`.

Postfix evaluation pops the right operand first

1. Scan tokens left to right. Push numeric values.
2. For a binary operator, pop the right operand b first, then the left operand a. Push the result of `a op b`.
3. Reject if fewer than two values are available. At the end exactly one value must remain.

For `8 3 2 * -`, the stacks are `[8]`, `[8,3]`, `[8,3,2]`, `[8,6]`, `[2]`. Division additionally requires zero-divisor and arithmetic-range checks.

For prefix, scan backward and preserve operand roles

- Scan prefix right to left. At an operator, the first popped value is the left operand and the second is the right operand.
- For `- 8 * 3 2`, first compute $3 \times 2 = 6$, then $8 - 6 = 2$.
- Each token is pushed and popped at most a constant number of times. Infix-to-postfix conversion and prefix or postfix evaluation therefore take O(t), for t tokens.
- Auxiliary stack space is O(t) in the worst case. Fixed-range arithmetic is treated as constant time. Arbitrary-precision arithmetic needs a separate cost model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Make the pop condition explicit

The input grammar contains nonnegative integer tokens, binary `+ - * / ^`, and parentheses. Here ^ means **right-associative exponentiation**, not C++ XOR. Other operators associate left. Unary minus and negative numeric tokens are outside this input grammar.

Category	Precedence	Associativity
+, −	1	left
*, /	2	left
^	3	right

```
must_pop(top, incoming):
    return top != '(' and
       (prec(top) > prec(incoming) or
        (prec(top) == prec(incoming) and incoming is left-associative))
```

Test **incoming's associativity**. Parentheses use a separate matching rule rather than ordinary binary precedence.

Infix to postfix: process one token at a time

output holds determined postfix fragments; ops holds pending operators. need_operand rejects adjacent values, empty parentheses and missing right operands.

```
output = []; ops = []; need_operand = true
for token in tokens:
    if token is a number:
        require need_operand; output.append(token); need_operand=false
    else if token == '(':
        require need_operand; ops.push('(')
    else if token == ')': close_group()
    else if token is binary operator op:
        require not need_operand
        while ops is not empty and must_pop(ops.top,op):
            output.append(ops.pop())
        ops.push(op); need_operand=true
    else: reject
```


A right parenthesis drains only its local group

```
close_group():
    require not need_operand
    while ops is not empty and ops.top != '(': output.append(ops.pop())
    require ops is not empty; ops.pop()   // discard matching left parenthesis
    need_operand=false
```

The helper shares output, ops and need_operand. It sets need_operand=false on success. Empty groups, missing operands and unmatched right parentheses all reject. Parentheses are never emitted.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

End of input is part of the algorithm

Continue with the same output, ops and need_operand from the preceding page:

```
require not need_operand                    // EOF: reject empty or incomplete
input
while ops is not empty:
    require ops.top != '('                  // reject unmatched left parenthesis
    output.append(ops.pop())
return output
```

Every failed requirement rejects the complete input. A partial output is not a successful conversion. Repeated popping is essential: one incoming operator may require several earlier operators to be emitted.

Even 9−4 needs an EOF drain

`9-4`

Token	Operator stack, bottom to top	Output
`9`	∅	`9`
`-`	`-`	`9`
`4`	`-`	`9 4`
`EOF`	∅	`9 4 -`

Before EOF the output is only `9 4`. Draining gives `9 4 -`, with value 5. Prefix is `- 9 4`.

Parentheses finish their local subexpression

`(9-4)*(2+3)`

Token	Operator stack, bottom to top	Output
`(`	`(`	`∅`
`9`	`(`	`9`
`-`	`( -`	`9`
`4`	`( -`	`9 4`
`)`	`∅`	`9 4 -`

The right parenthesis finishes 9−4, emitting `9 4 −`. ops is empty; multiplication and the second group follow.

Finish the second group, then drain multiplication

`(9-4)*(2+3)`

Token	Operator stack, bottom to top	Output
`*`	`*`	`9 4 -`
`(`	`* (`	`9 4 -`
`2`	`* (`	`9 4 - 2`
`+`	`* ( +`	`9 4 - 2`
`3`	`* ( +`	`9 4 - 2 3`
`)`	`*`	`9 4 - 2 3 +`
`EOF`	`∅`	`9 4 - 2 3 + *`

Postfix stack states are `[9]→[9,4]→[5]→[5,2]→[5,2,3]→[5,5]→[25]`. Prefix `* - 9 4 + 2 3` also evaluates to 25.

Trace repeated popping and right association

`9-4*2/1+3`

Token	Operator stack, bottom to top	Output
9	∅	9
-	-	9
4	-	9 4
*	- *	9 4
2	- *	9 4 2
/	- /	9 4 2 *
1	- /	9 4 2 * 1
+	+	9 4 2 * 1 / -
3	+	9 4 2 * 1 / - 3
EOF	∅	9 4 2 * 1 / - 3 +

Equal precedence: inspect incoming associativity

At +, pop / and then the equal-precedence left-associative -. Popping once is insufficient. The value is 4. For right-associative `2^3^2`, do not pop on the second ^. Postfix is `2 3 2 ^ ^`, giving 512 rather than 64.

Right association gives 2^(3^2)=2^9=512. Left association gives (2^3)^2=8^2=64. These are different expression trees, not interchangeable evaluation orders.

Reject invalid syntax and unsafe arithmetic

Input	Rejection point	Reason
`(9-4`	EOF	A left parenthesis remains after emitting -
`9-4)`	At)	No matching left parenthesis
`9-`	EOF	need_operand remains true
`9 4`	Second number	Missing binary operator
`()`	At)	Missing operand inside parentheses

Do not infer unary versus binary syntax from stack size. Evaluation separately rejects division by zero and fixed-integer overflow.

Why the invariant and linear bound hold

An unmatched left parenthesis separates stack segments. Within each segment, precedence is nondecreasing from bottom to top. Equal-precedence operators can remain together when waiting for right association. Output contains the postfix order of completed subexpressions; stacked operators await their right operand or a higher-precedence subexpression.

Repeated popping emits exactly the operators that must finish before the new operator. A right parenthesis finishes its entire local segment while preserving outside operators. At EOF no further operand can arrive, so every pending operator must be emitted. A remaining left parenthesis has no valid mate and must cause rejection, not appear as an output operator.

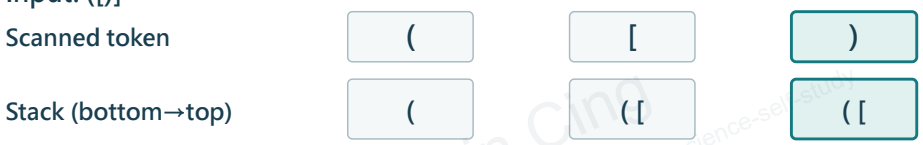
Each operator and parenthesis is pushed and popped at most once. A single token can trigger multiple pops, but the complete t-token conversion uses O(t) time and O(t) space. This is an aggregate proof, not constant worst-case work per token. Unary operators, functions, negative numeric tokens, and special floating-point values require an expanded grammar and proof.

Bracket matching: retain the kinds still open

The input alphabet is `()[]{}`. Push each opening bracket. On a closing bracket, reject if the stack is empty or its top is not the matching opener; otherwise pop. Accept only if the stack is empty at the end. If other characters are allowed, explicitly specify whether to ignore or reject them.

Bracket matching: the closer must match the stack top

Input: ([)]



On), the top is [, which expects].

Reject immediately; equal bracket counts do not guarantee nesting.

For ([]), the stack is empty after the scan, so accept.

The heavy border marks the mismatch independently of color.

Equal counts do not ensure correct nesting

`([)]` has equal counts of each opening and closing kind, but at the third character `)` the stack, bottom to top, is `(`, `[`. The top expects `]`, so reject. For `([])`, the successive states are `[ ( ]`, `[ ( , [ ]`, `[ ( ]`, `[]`, and the input is accepted.

Invariant: after every accepted prefix, the stack contains exactly its unmatched opening brackets in order, with the next one to close on top. Each symbol causes at most one push or pop: $O(n)$ time and $O(n)$ auxiliary space. The empty string is accepted under this contract.

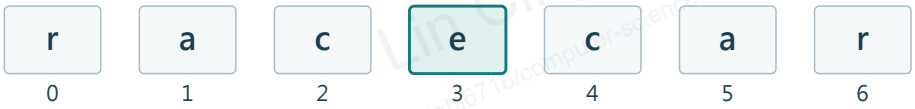
Palindrome with known length: reverse the first half

The input is a sequence S with known length n. Push its first $\lfloor n/2 \rfloor$ symbols in order. If n is odd, skip the middle symbol. Scan the second half left to right, comparing and popping the top each time. Reject on the first mismatch and accept only if all pairs match.

For `racecar`, push `r,a,c`, skip `e`, then compare `c,a,r` against successive tops. The empty and one-symbol sequences are accepted. Specify the symbol unit: bytes, Unicode code points, and grapheme clusters may give different results.

Palindrome: push the first half and compare the second

racecar: length 7; skip the middle e



Push r,a,c: the stack bottom→top is [r,a,c].

Skip e; compare c, a, r in order. All three match.

Pop after each comparison; accept after all symbols match.

Empty and one-symbol inputs are accepted; length is known.

Palindrome correctness and cost

After the first half is pushed, the top is immediately left of the middle. Pop number k compares the two positions symmetric about the middle. Every pair matches exactly when the sequence equals its reversal. Time is $O(n)$ and the stack holds at most $\lfloor n/2 \rfloor$ symbols.

This version knows the length and can scan the corresponding positions. A one-way stream of unknown length cannot identify its midpoint before reaching it without a different input assumption or algorithm.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Recognizing $a^n b^n$: count and order both matter

Define $L = \{a^n b^n \mid n \geq 1\}$ over the alphabet `a`,`b`. Read at least one `a`, pushing one marker per `a`. On the first `b`, enter the second phase and pop a marker per `b`. Reject an `a` in phase two, any other symbol, or a `b` read with an empty stack. At the end, accept only if both `a` and `b` have appeared and the stack is empty.

`aabb` is accepted; `aab` leaves one marker; the second `b` in `abb` has no marker; and `abab` has an `a` in phase two. The empty string is excluded here because $n \geq 1$; defining $n \geq 0$ would include it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

3.6 Stack permutations do not allow every output order

Input arrives in fixed order 1,2,3. Only push the next input value or pop the current top to output. No bypass is allowed.

Target	Forced operations	Result
2,1,3	push 1,push 2,pop 2,pop 1,push 3,pop 3	Feasible; maximum depth 2
3,1,2	To output 3 first: push 1,push 2,push 3,pop 3	Impossible: top 2 blocks the required 1

For each target x, push unread inputs until top=x. If input is exhausted and top still differs, reject; otherwise pop x. **The choices are forced**: when top is not x, popping would output a wrong value, so pushing the next input is the only possible progress. Each distinct value enters and leaves at most once: O(n) time and O(n) space.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why are there five outputs for n=3, rather than six?

Write U for push and D for pop. A legal sequence has n of each, with at least as many U as D in every prefix. This is exactly a balanced-parenthesis sequence, with U as an opening parenthesis.

Match the first push with the pop that removes it, obtaining `U A D B`. If A has k pushes, B has n−1−k. Both are legal, and the decomposition is unique:

$$C_0 = 1, \qquad C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}.$$

Thus C_1=1, C_2=2, C_3=2+1+2=5 and C_4=5+2+2+5=14. For a fixed distinct input, each output forces its operation sequence, as proved earlier. We therefore count distinct feasible outputs as well as operations.

Catalan's formula subtracts the illegal paths

Ignoring the prefix restriction gives $\binom{2n}{n}$ U/D sequences. For an illegal path, swap U and D through its first visit to height−1. The original prefix had one more D, so the transformed complete path has n+1 U and n−1 D.

Invert this transformation by swapping the prefix through the first visit to height+1. The inverse recovers exactly one illegal path; hence the correspondence is bijective. There are $\binom{2n}{n-1}$ illegal paths, giving

$$C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{n+1}\binom{2n}{n}.$$

For n=0 there is one empty operation sequence. Distinct values, fixed input order and no bypass are essential to this output count. Repeated values need a separate count of distinct outputs.

References and attribution

Lin Cing, handwritten data structures notes; C++ draft stack and queue; MIT 6.006, Lecture Notes 9–10.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study